

1 Lesson 01-10-26

Last lesson we started talking about *memories*.

1.0.1 Digital memories

The kinds of memory we mainly care about are **digital** memories. These are **ordered** collections of data, where not all data is necessarily available or accessed concurrently.

Data, in our sense, is made out of n -bit *words*:

- A memory contains N_{word} words of identical size;
- Each word as a width of n_{bits} ;
- Each bit is a binary storage element (such as a flip-flop, even though flip-flops are usually not used in digital memories).

Words are contiguous in memory, and each word is identified by a unique index we call **address**. Addresses usually start from 0 up to $A_{\text{last}} = N_{\text{words}} - 1$.

Sizing

Since we usually address with binary counters, the number of words is always a power of 2: addresses can be expressed on:

$$k = \log_2(N_{\text{words}}) \text{ bits}$$

The number of bits in a word, on the other hand, can be any number (though usually it also is a power of 2, allowing memories to store their own addresses). Sometimes it is useful to include extra bits after 2^n bits, for parity checking or other **EDC** (*Error Detection Code*) techniques.

We can call the **size** of the whole memory the product of words per bits:

$$\text{size} = N_{\text{words}} \times n_{\text{bits}}$$

This can be interpreted as the total number of bits in the memory.

Two memories can have the same size but different organization: for instance, the same total number of bits, but different word size and number of words. The organization affects the memory interface (number and kind of access pins).

We shall also note that *external* access also affects the memory interface:

- Parallel, Serial;
- Random, Sequential;
- Special access modes.

1.0.2 Sense amplifiers

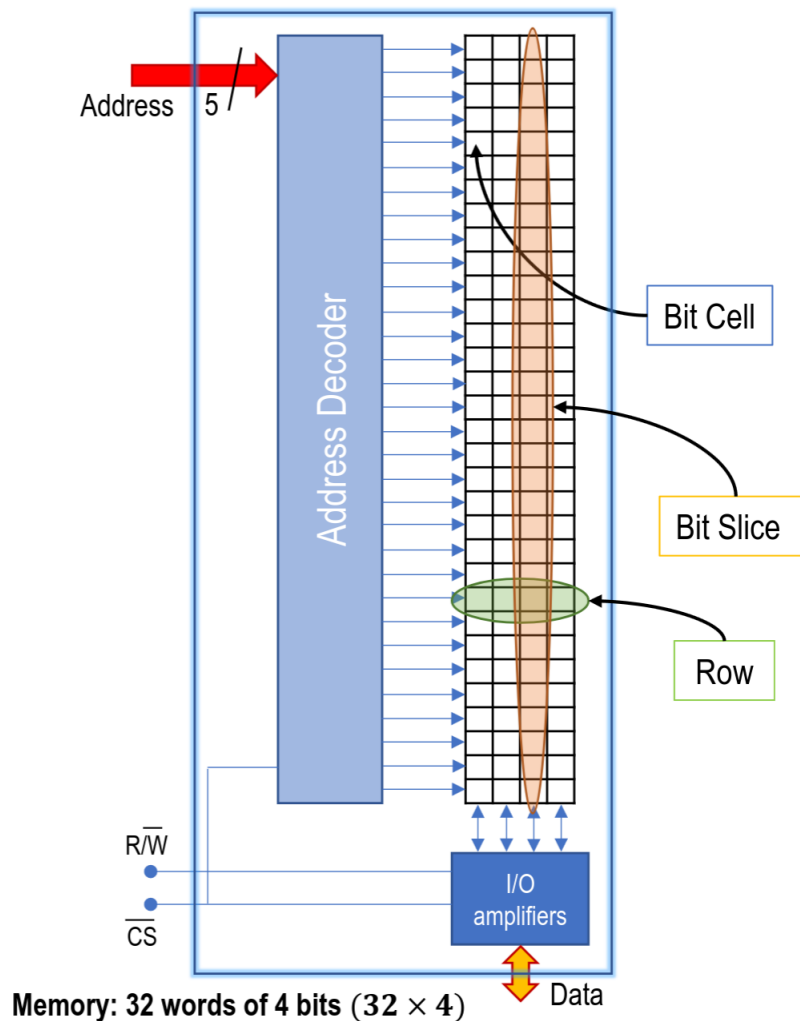
We also quickly explain what *sense amplifiers* are: the signals stored within memories are usually too weak to interface to the external word. For this reason we use amplifiers. These are used to compare a *bit line* (more later) to a *reference line*, and get a biased signal. Note that the reference line can be another bit line!

1.0.3 Logical structure of digital memories

The **logical** structure of a memory, as we explained it, is an **array** of bit cells:

- One row for each word;

- One bit slice for each bit in the word.



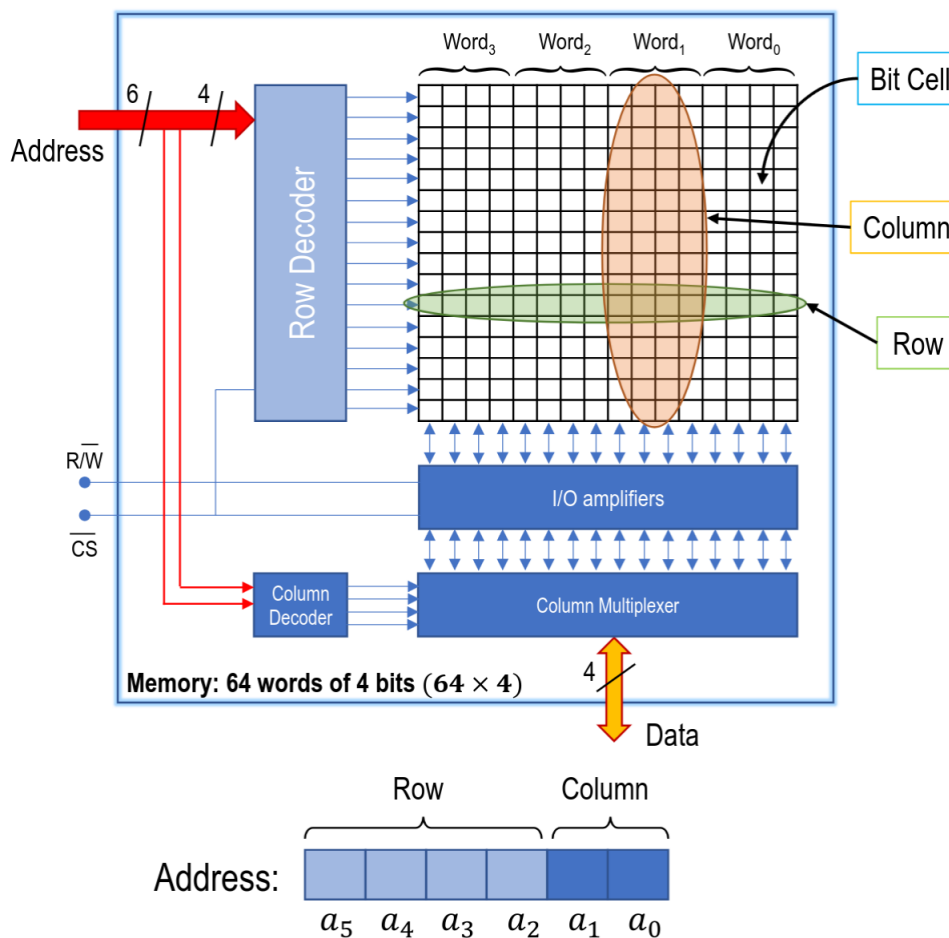
Some *address decoder* is used to select a single row of the memory: from k address lines to 2^k word select lines. At the word interface, a sense amplifier and I/O circuit are used to interface to the external world.

This is not the ideal approach though, as:

- The aspect ratio is unusual and difficult to work with;
- The size of address decoder is exponential on the number of address bits;
- The memory is *slow*: large parasitic capacitance on bit lines increases propagation delays.

1.0.4 Physical structure of digital memories

We then move on to defining the **physical** structure of a real memory. The idea is to not have a *linear*, but a **square** array (or matrix) of word cells.



So we find that the real structure is still an array of bit cells, organized in a square array (optimal aspect ratio). This means that we can have more than one word for each row: one bit slice for each bit in the word, and one column for each word.

Our address decoder has to become a *row decoder*, and we need an additional *column multiplexer* after the I/O amplifiers to select the correct word from a row. As we can see, this is achieved by splitting the address:

- The *upper* side of the address is used to select a row;
- Within a row, the *lower* side of the address is used to select a column.

The sense amplifier and I/O circuit are still used to interface to the external world.

From the user interface, the interface is the same, but the internal organization is different (and way easier to implement physically).

1.1 Memory interfaces

We can now talk about the **interfaces** of memories. Usually these are device pins used to access the memory content, write/read it, ... They are grouped based on their function:

- **Address bus:** identifies the memory location (the index of the word) where an operation is performed. A straightforward address bus is comprised of $k = \log_2(N_{\text{words}})$ lines;
- **Data bus:** provides the data to/from the memory word. Can be bidirectional, or split in input/output monodirectional buses. A straightforward data bus is comprised of n_{bits} lines;

- **Control bus:** this contains the control lines used to perform operations on the data stored in memory. A lot of control lines can be defined, for instance:
 - Clock;
 - Reset;
 - Chip select;
 - Output enable;
 - Write enable;
 - Read/write select;
 - ...

Memory interfaces (as the name suggests) are designed for interfacing a memory to a digital circuit. The digital circuit shall connect to all pins of the memory, drive the inputs according to the specifications in the datasheet, and read the outputs. All of this must be done while complying with the timing diagrams.

- In a *custom* digital circuit, the interface is usually designed specifically for the given memory;
- In a *standard* digital circuit, there is a generic memory interface: it may require external components to correctly drive a certain memory.

1.1.1 Memory operations

This can be a good moment to note that the memory **operations** available are:

- Read: given an address, returns the word contained in memory at that address;
- Write: Given a word and an address, stores the word in memory at that address;
- Match: Given a word, searches the memory for that particular word content and returns the address. This is specific of Content Addressable Memories (**CAM**, *Content Addressable Memory*, used in caches, for instance);
- Erase: initializes the memory to a pre-defined content. Typical of some memories;
- Other operations are sometimes available, such as: Burst operation, Partial operation, Precharge, Refresh, ...

1.1.2 Memory protocol

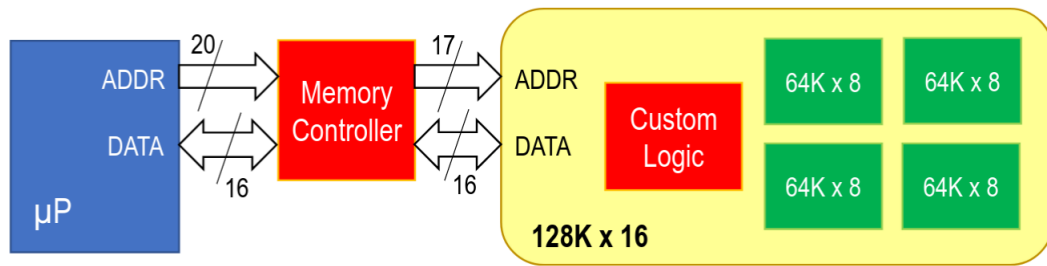
Data access via the address, data and control lines is performed by an external component on the memory via a predetermined **protocol**. The protocol allows organizing the memory to have better performance and area size:

- It defines the sequences of signal activations to correctly perform different operations;
- It defines the timing constraints.

A memory protocol is often described as a timing diagram in the device datasheet.

1.1.3 Memory composition

One of the most important concepts regarding memories at the logic level is memory **composition**. This entails combining multiple memory chips of a given size s_i to obtain a single addressable memory of size $\sum_i s_i$.



We have 3 main cases:

1. Let's assume the case where the required memory needs more words than those available in a single memory chip. For compatibility, we also assume that word width is the same.

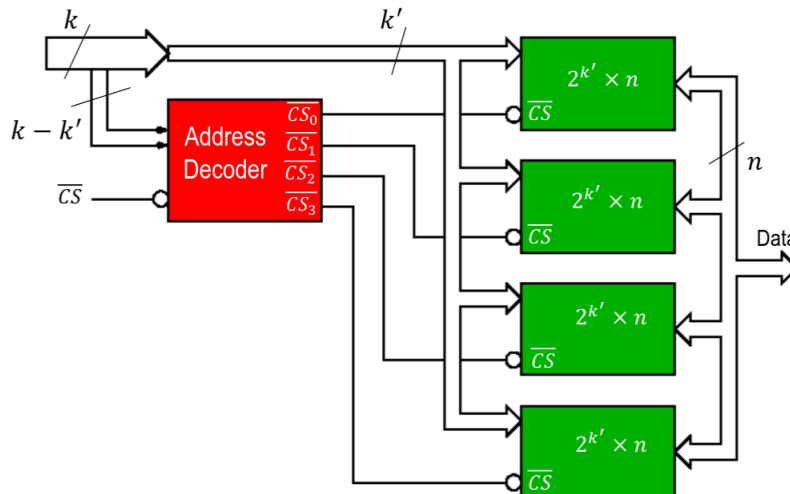
This means that addresses are size k , with each memory chip having addresses of size k' and:

$$k > k'$$

The number of memory chips needed is given by:

$$n_{\text{chips}} = \frac{2^k}{2^{k'}} = 2^{k-k'}$$

We want only one of these chips to be active (to *respond*) for a given address range: this means implementing custom logic to drive the $\overline{CS}$ (active low Chip Select) signal of the memory chips. This custom logic is basically a decoder, controlled by the $k - k'$ additional address lines (over k') and enabling $2^{k-k'}$ (which have to alternatively drive one singular data output bus, a.k.a. tristate outputs).



2. Now there's the case where the required memory needs words with more bits than those available on a single memory chip. For compatibility, we also assume that N_{word} is the same.

This means that words are to be size n , with each memory chip having words of size n' and:

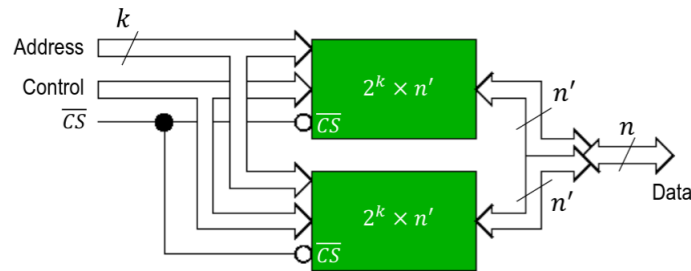
$$n > n'$$

The number of memory chips needed is given by:

$$n_{\text{chips}} = \text{ceil} \left(\frac{n}{n'} \right)$$

At this point the composition is trivial:

- All memory chips share the same address,;
- All memory chips share the same $\overline{CS}$ chip select signal;
- Their data busses are combined (concatenated) to form a larger data bus.



3. The last case is the combination of the first two: required memory needs more words than those available in a single memory chip and needs more bits in a word than those available in a single memory chip.

This means that there are: • k required address lines, but k' address lines in a memory chip, with $k > k'$; • n required data lines, but n' data lines in a memory chip, with $n > n'$.

We need more than one memory chip (of course), and exactly:

$$n_{\text{chips}} = \text{ceil} \left(\frac{n}{n'} \right) \cdot 2^{k-k'}$$

1.1.4 Memory controllers

Lastly, let's spend a note on **memory controllers**. These can be useful, for instance, when general purpose processors are interfaced with off-the-shelf memory chips: the amount of address lines might not match!

Now in this situation, without a memory controller, the microprocessor accesses the same location in memory with multiple addresses. Also, some pins are left floating.

A *memory controller* is used to translate addresses and activate the memory. The memory is mapped to a certain address range in the microprocessor's addressable space: the range does not necessarily start at address 0: the most significant bits can be used by the memory controller to select the appropriate memory. Though, it's good to say that the address range is usually aligned to a power of 2 to simplify the logic.

We know this far too well from our experience with *compiler* tooling: the compiler for the microprocessor should know the chosen address range, so that it generates code that correctly accesses the memory.

Lastly, we note that the memory controller may be integrated inside the microprocessor.